RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence
V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 3b: Model Planning and Building – Regression

Name: Nischal **Date:** 06-10-2026

Register Number: 241801188 **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Dataset: Advertising Dataset (Kaggle) – 200 records: TV, Radio, Newspaper, Sales

Aim

To plan, build and evaluate regression models that predict product Sales from the advertising spending on TV, Radio and Newspaper: explore the Advertising dataset, study the relationships between the features, build Linear, Polynomial (degree 2) and Support Vector (RBF kernel) regression models, compare them using MAE, MSE, RMSE and R^2, identify the model with the lowest prediction error, and use it to suggest how the advertising budget can be optimised.

Task 1: Load the Advertising dataset

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.compose import TransformedTargetRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='whitegrid')
pd.set_option('display.max_columns', None)

df = pd.read_csv('advertising.csv')
print("Advertising dataset loaded successfully")
```

Out [1]:

```
Advertising dataset loaded successfully
```

Task 2: Explore the dataset

2.1 First and last five records

In [2]:

```
df.head()
```

Out [2]:

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

5 rows × 4 columns

In [3]:

```
df.tail()
```

Out [3]:

	TV	Radio	Newspaper	Sales
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

5 rows × 4 columns

2.2 Shape and data types

In [4]:

```
print("Shape of the dataset:", df.shape)
print("\nData types:")
print(df.dtypes)
```

Out [4]:

```
Shape of the dataset: (200, 4)

Data types:
TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object
```

2.3 Missing values and duplicate records

In [5]:

```
print("Missing values in each column:")
print(df.isnull().sum())
print("\nTotal missing values  :", int(df.isnull().sum().sum()))
print("Duplicate records     :", int(df.duplicated().sum()))
```

Out [5]:

```
Missing values in each column:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64

Total missing values  : 0
Duplicate records     : 0
```

> **Observation:** The dataset has 200 records and 4 columns, all numeric, with no missing values and no duplicate records, so no cleaning is needed.

2.4 Summary statistics

In [6]:

```
df.describe().round(2)
```

Out [6]:

	TV	Radio	Newspaper	Sales
count	200.00	200.00	200.00	200.00
mean	147.04	23.26	30.55	15.13
std	85.85	14.85	21.78	5.28
min	0.70	0.00	0.30	1.60
25%	74.38	9.98	12.75	11.00
50%	149.75	22.90	25.75	16.00
75%	218.82	36.52	45.10	19.05
max	296.40	49.60	114.00	27.00

8 rows × 4 columns

Task 3: Select the input features (X) and target variable (y)

In [7]:

```
features = ['TV', 'Radio', 'Newspaper']
X = df[features]
y = df['Sales']

print("Input features (X):", features, "->", X.shape)
print("Target variable (y): Sales ->", y.shape)
```

Out [7]:

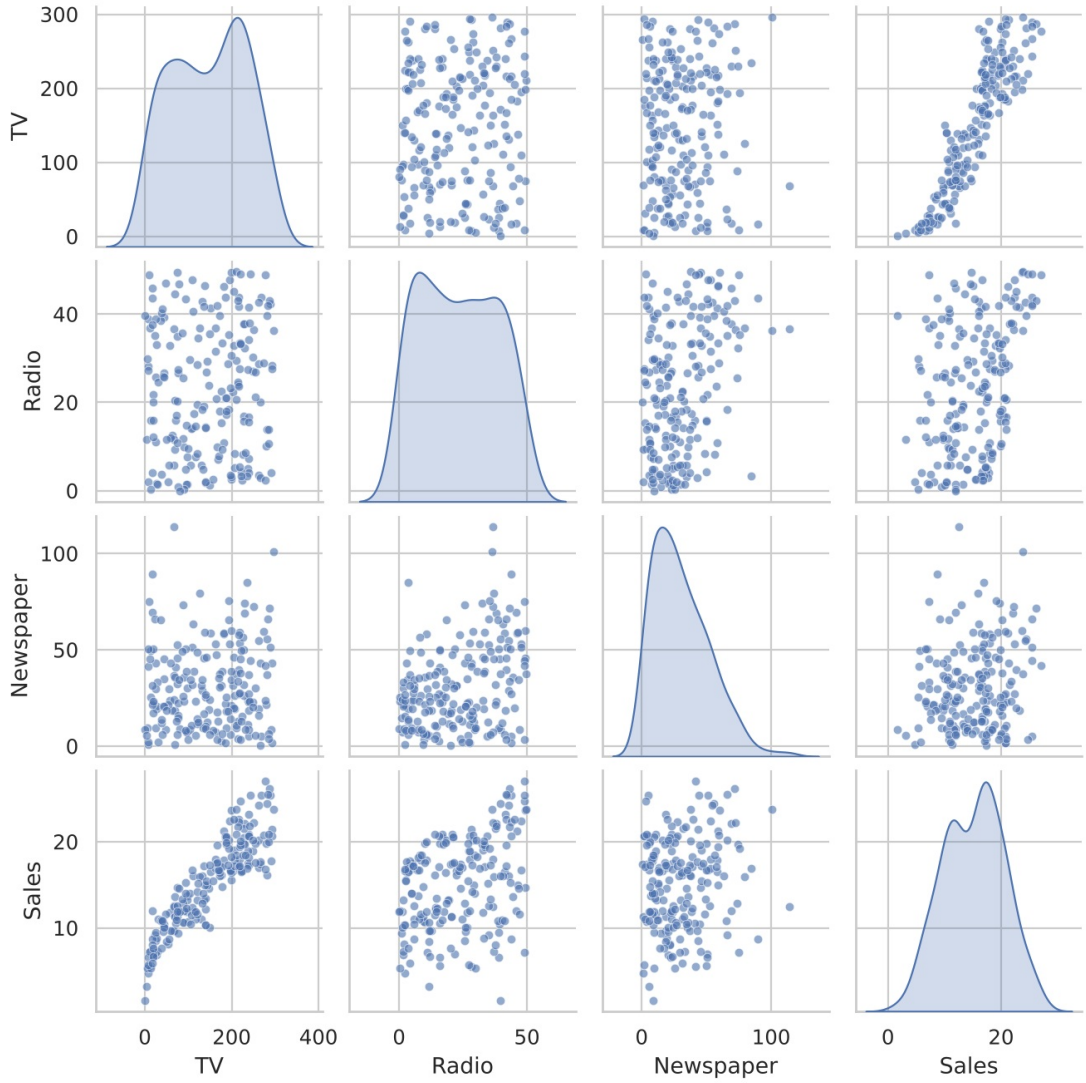
```
Input features (X): ['TV', 'Radio', 'Newspaper'] -> (200, 3)
Target variable (y): Sales -> (200,)
```

Task 4: Pair plot of the features

In [8]:

```
sns.pairplot(df[['TV', 'Radio', 'Newspaper', 'Sales']], height=2.1,
             diag_kind='kde', plot_kws={'alpha': 0.6, 's': 22})
plt.show()
```

Out [8]:



Observation: Sales rises steadily with TV spending and the points follow a clear upward line. The Radio panel shows a weaker upward trend, and the Newspaper panel is a loose cloud with almost no pattern. The spread of Sales also widens at high TV spending, which hints that the relationship is not perfectly linear.

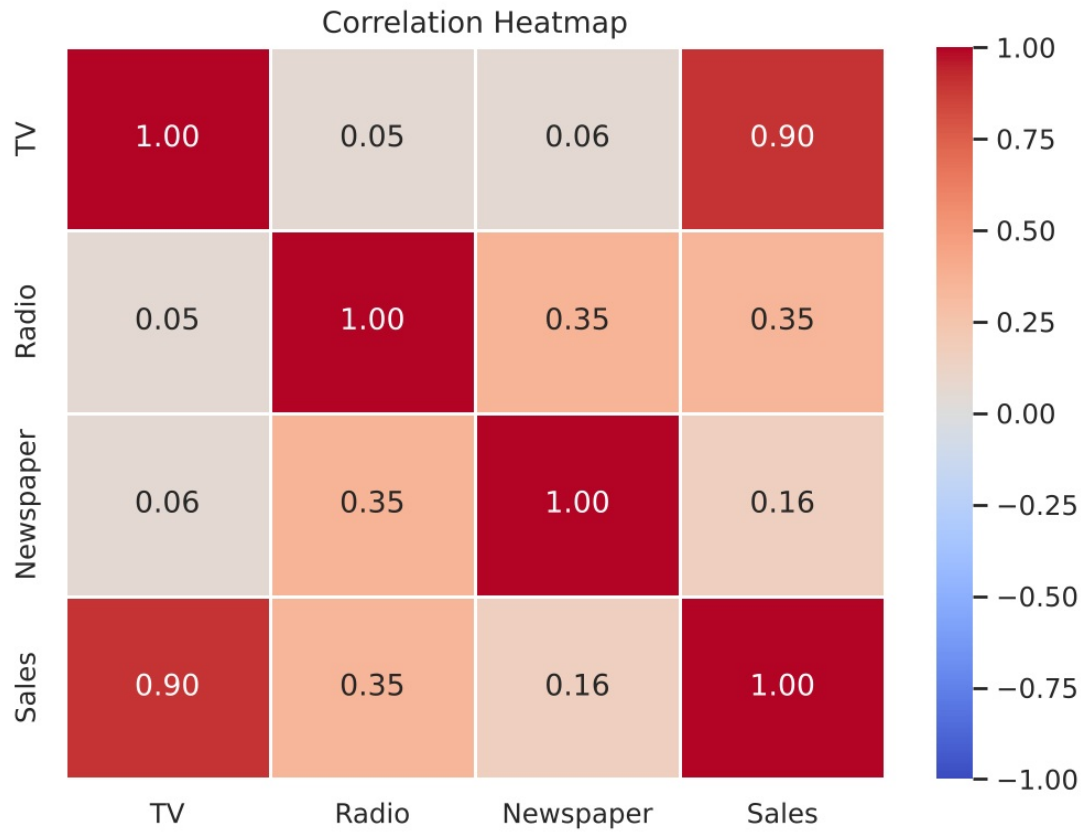
Task 5: Correlation heatmap

In [9]:

```
corr = df.corr()

plt.figure(figsize=(6.5, 5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0, linewidths=0.5)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Out [9]:



Task 6: Feature most strongly correlated with Sales

In [10]:

```
sales_corr = corr['Sales'].drop('Sales').sort_values(ascending=False)
print("Correlation of each feature with Sales:")
print(sales_corr.round(3))
print("\nMost strongly correlated feature:", sales_corr.idxmax(), f"(r = {sales_corr.max():.3f})")
```

Out [10]:

```
Correlation of each feature with Sales:
TV           0.901
Radio        0.350
Newspaper    0.158
Name: Sales, dtype: float64

Most strongly correlated feature: TV (r = 0.901)
```

> **Result: TV** is the feature most strongly correlated with Sales (r = 0.901), followed by Radio (r = 0.350). Newspaper has the weakest link (r = 0.158). Radio and Newspaper are themselves correlated (r = 0.354), so part of Newspaper's small correlation with Sales may only reflect Radio spending.

Task 7: Split the dataset into Training (80%) and Testing (20%)

In [11]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set :", X_train.shape, "->", round(len(X_train) / len(X) * 100, 1), "% of the data")
print("Testing set  :", X_test.shape, "->", round(len(X_test) / len(X) * 100, 1), "% of the data")
```

Out [11]:

```
Training set : (160, 3) -> 80.0 % of the data
Testing set  : (40, 3) -> 20.0 % of the data
```

> **Note:** `random_state=42` makes the split repeatable, so all three models are trained and tested on exactly the same rows.

Task 8: Linear Regression

In [12]:

```
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
y_pred_lin = lin_model.predict(X_test)

print("Intercept:", round(lin_model.intercept_, 4))
for name, coef in zip(features, lin_model.coef_):
    print(f"Coefficient of {name:9s}: {coef:.4f}")

print("\nFirst 10 test records:")
display(pd.DataFrame({'Actual Sales': y_test.values[:10], 'Predicted Sales': y_pred_lin[:10].round(2)}).T)
```

Out [12]:

```
Intercept: 4.7141
Coefficient of TV       : 0.0545
Coefficient of Radio    : 0.1009
Coefficient of Newspaper: 0.0043

First 10 test records:
```

	0	1	2	3	4	5	6	7	8	9
Actual Sales	16.90	22.40	21.40	7.30	24.70	12.60	22.30	8.40	16.50	16.10
Predicted Sales	17.03	20.41	23.72	9.27	21.68	12.57	21.08	8.69	17.24	16.67

2 rows × 10 columns

> **Observation:** Holding the other channels fixed, one more unit of spending is linked to 0.0545 more sales for TV, 0.1009 for Radio and only 0.0043 for Newspaper, which is close to zero. Radio gives more sales per unit of spending than TV, but TV is the channel with the largest spending range.

Task 9: Polynomial Regression (degree = 2)

In [13]:

```
poly_model = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), LinearRegression())
poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)

poly_names = poly_model.named_steps['polynomialfeatures'].get_feature_names_out(features)
poly_coefs = pd.Series(poly_model.named_steps['linearregression'].coef_, index=poly_names)
print("Number of polynomial features:", len(poly_names))
print("\nCoefficients of the degree-2 model:")
print(poly_coefs.round(5))

print("\nFirst 10 test records:")
display(pd.DataFrame({'Actual Sales': y_test.values[:10], 'Predicted Sales': y_pred_poly[:10].round(2)}).T)
```

Out [13]:

```
Number of polynomial features: 9

Coefficients of the degree-2 model:
TV                0.07622
Radio            -0.03198
Newspaper        -0.00192
TV^2             -0.00011
TV Radio          0.00042
TV Newspaper     -0.00003
Radio^2           0.00145
Radio Newspaper   0.00016
Newspaper^2       0.00009
dtype: float64

First 10 test records:
```

	0	1	2	3	4	5	6	7	8	9
Actual Sales	16.90	22.40	21.40	7.30	24.70	12.60	22.30	8.40	16.50	16.10
Predicted Sales	17.56	21.96	22.08	7.35	23.24	12.37	22.13	8.77	17.01	16.91

2 rows × 10 columns

> **Observation:** Degree 2 expands the three inputs into 9 terms (the original features, their squares and the products between them). The positive `TV Radio` product term (0.00042) means TV and Radio spending work better together than each alone, and the negative TV^2 term (-0.00011) shows that extra TV spending gradually adds less.

Task 10: Support Vector Regression (RBF kernel)

SVR with the RBF kernel depends on distances between points, so the inputs are standardised first. The target (Sales) is also standardised so that the default SVR settings are on a suitable scale; predictions are converted back to the original Sales units. The default hyper-parameters (`C = 1`) are used.

In [14]:

```
svr_model = TransformedTargetRegressor(
    regressor=make_pipeline(StandardScaler(), SVR(kernel='rbf')),
    transformer=StandardScaler())
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

print("SVR (RBF kernel) trained.")
print("Number of support vectors:", len(svr_model.regressor_.named_steps['svr'].support_), "of", len(X_train), "training points")
print("\nFirst 10 test records:")
display(pd.DataFrame({'Actual Sales': y_test.values[:10], 'Predicted Sales': y_pred_svr[:10].round(2)}).T)
```

Out [14]:

```
SVR (RBF kernel) trained.
Number of support vectors: 106 of 160 training points

First 10 test records:
```

	0	1	2	3	4	5	6	7	8	9
Actual Sales	16.90	22.40	21.40	7.30	24.70	12.60	22.30	8.40	16.50	16.10
Predicted Sales	16.65	21.71	20.73	8.95	22.45	12.06	22.45	8.34	17.39	16.43

2 rows × 10 columns

Task 11: Model evaluation

In [15]:

```
predictions = {'Linear Regression': y_pred_lin,
               'Polynomial Regression (deg 2)': y_pred_poly,
               'SVR (RBF)': y_pred_svr}

rows = []
for name, pred in predictions.items():
    mse = mean_squared_error(y_test, pred)
    rows.append({'Model': name,
                 'MAE': mean_absolute_error(y_test, pred),
                 'MSE': mse,
                 'RMSE': np.sqrt(mse),
                 'R2 Score': r2_score(y_test, pred)})

results = pd.DataFrame(rows).set_index('Model').round(4)
display(results)
```

Out [15]:

Model	MAE	MSE	RMSE	R2 Score
Linear Regression	1.2748	2.9078	1.7052	0.9059
Polynomial Regression (deg 2)	0.9034	1.4425	1.2011	0.9533
SVR (RBF)	1.1851	2.4403	1.5622	0.9210

3 rows × 4 columns

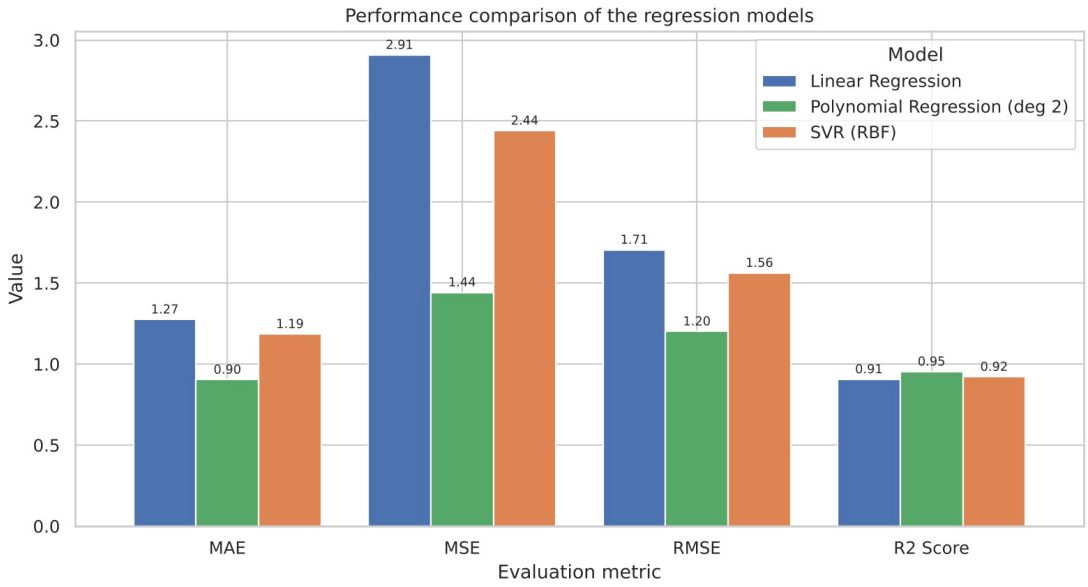
> **Observation:** All three models explain most of the variation in Sales (R^2 between 0.91 and 0.95). The Polynomial Regression (deg 2) has the smallest errors on every measure (MAE 0.9034, RMSE 1.2011) and the highest R^2 (0.9533). On the test data, its typical error is about 1.20 sales units, against 1.71 for Linear Regression.

Task 12: Compare the models (grouped bar chart)

In [16]:

```
ax = results.T.plot(kind='bar', figsize=(10, 5.5), width=0.8,
                    color=['#4C72B0', '#55A868', '#DD8452'])
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', fontsize=8, padding=2)
ax.set_title('Performance comparison of the regression models')
ax.set_xlabel('Evaluation metric')
ax.set_ylabel('Value')
plt.xticks(rotation=0)
plt.legend(title='Model')
plt.tight_layout()
plt.show()
```

Out [16]:



> **Note:** MAE, MSE and RMSE are errors, so lower is better; R^2 is a fit score, so higher is better. They share one chart as asked, so the R^2 bars (close to 1) are on a different scale from the error bars.

Task 13: Actual vs Predicted Sales for the best-performing model

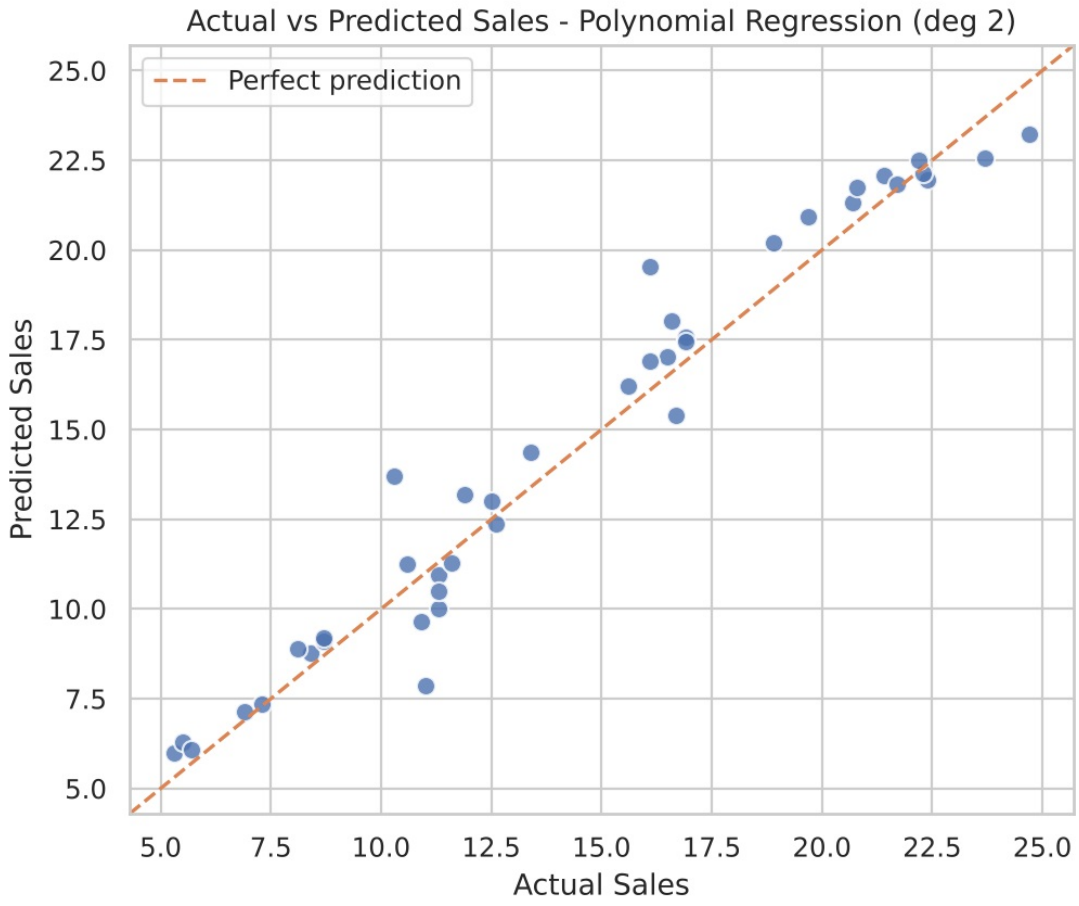
In [17]:

```
best_name = results['RMSE'].idxmin()
best_pred = predictions[best_name]
models = {'Linear Regression': lin_model, 'Polynomial Regression (deg 2)': poly_model, 'SVR (RBF)': svr_model}
best_model = models[best_name]

plt.figure(figsize=(6.5, 5.5))
plt.scatter(y_test, best_pred, color='#4C72B0', alpha=0.8, edgecolor='white', s=60)
lims = [min(y_test.min(), best_pred.min()) - 1, max(y_test.max(), best_pred.max()) + 1]
plt.plot(lims, lims, color='#DD8452', linestyle='--', label='Perfect prediction')
plt.xlim(lims); plt.ylim(lims)
plt.title(f'Actual vs Predicted Sales - {best_name}')
plt.xlabel('Actual Sales')
plt.ylabel('Predicted Sales')
plt.legend()
plt.tight_layout()
plt.show()

print("Best model:", best_name)
```

Out [17]:




```
Best model: Polynomial Regression (deg 2)
```

> **Observation:** The points lie close to the dashed line of perfect prediction, so the Polynomial Regression (deg 2) predicts Sales for unseen records accurately, with no obvious bias toward over- or under-prediction.

Task 14: Model with the lowest prediction error

In [18]:

```
for metric in ['MAE', 'MSE', 'RMSE']:
    print(f"Lowest {metric:4s}:", results[metric].idxmin(), f"({results[metric].min():.4f})")
print("Highest R2  :", results['R2 Score'].idxmax(), f"({results['R2 Score'].max():.4f})")

print("\nRanking by RMSE (lowest error first):")
display(results.sort_values('RMSE'))
```

Out [18]:

```
Lowest MAE : Polynomial Regression (deg 2) (0.9034)
Lowest MSE : Polynomial Regression (deg 2) (1.4425)
Lowest RMSE: Polynomial Regression (deg 2) (1.2011)
Highest R2  : Polynomial Regression (deg 2) (0.9533)

Ranking by RMSE (lowest error first):
```

Model	MAE	MSE	RMSE	R2 Score
Polynomial Regression (deg 2)	0.9034	1.4425	1.2011	0.9533
SVR (RBF)	1.1851	2.4403	1.5622	0.9210
Linear Regression	1.2748	2.9078	1.7052	0.9059

3 rows × 4 columns

> **Result:** The **Polynomial Regression (deg 2)** has the lowest prediction error, with MAE 0.9034, MSE 1.4425 and RMSE 1.2011, and the highest R^2 (0.9533). It reduces the RMSE by 29.6% compared with Linear Regression. The SVR comes second (RMSE 1.5622) and Linear Regression is last (RMSE 1.7052). The polynomial model wins because it captures the interaction between TV and Radio and the diminishing effect of TV, which a straight-line model cannot.

Task 15: Recommendations to optimise the advertising budget

The recommendations are based on the best model (Polynomial Regression, degree 2). To make them concrete, the model is used to predict Sales for a typical (average) budget and for a few changes to it. Spending changes stay within the range of the data, so the model is not asked to predict outside what it has seen.

In [19]:

```
avg = X.mean()
print("Average budget in the data:")
print(avg.round(2).to_string())

def predict_sales(tv, radio, newspaper):
    return float(best_model.predict(pd.DataFrame([[tv, radio, newspaper]], columns=features))[0])

base = predict_sales(avg['TV'], avg['Radio'], avg['Newspaper'])

scenarios = {
    'Average budget (baseline)':            (avg['TV'],      avg['Radio'],      avg['Newspaper']),
    'A. Add 15 to TV':                      (avg['TV'] + 15, avg['Radio'],      avg['Newspaper']),
    'B. Add 15 to Radio':                   (avg['TV'],      avg['Radio'] + 15, avg['Newspaper']),
    'C. Add 15 to Newspaper':               (avg['TV'],      avg['Radio'],      avg['Newspaper'] + 15),
    'D. Move 15 from Newspaper to Radio':   (avg['TV'],      avg['Radio'] + 15, avg['Newspaper'] - 15),
}
scenario_table = pd.DataFrame(
    [{'Scenario': k, 'TV': v[0], 'Radio': v[1], 'Newspaper': v[2], 'Predicted Sales': predict_sales(*v)}
     for k, v in scenarios.items()]).set_index('Scenario')
scenario_table['Change vs baseline'] = scenario_table['Predicted Sales'] - base
display(scenario_table.round(2))

# Does extra Radio help more when TV spending is high? (the TV x Radio interaction)
gain_low_tv  = predict_sales(50, 20 + 15, avg['Newspaper']) - predict_sales(50, 20, avg['Newspaper'])
gain_high_tv = predict_sales(250, 20 + 15, avg['Newspaper']) - predict_sales(250, 20, avg['Newspaper'])
print(f"Sales gain from adding 15 to Radio when TV = 50 : {gain_low_tv:.2f}")
print(f"Sales gain from adding 15 to Radio when TV = 250: {gain_high_tv:.2f}")

# Total effect of each channel across its full spending range (other channels at their average)
span = {}
for col in features:
    lo, hi = avg.copy(), avg.copy()
    lo[col], hi[col] = X[col].min(), X[col].max()
    span[col] = predict_sales(*hi) - predict_sales(*lo)
print("\nChange in predicted Sales across the full range of each channel:")
for col, v in span.items():
    print(f"  {col:10s}: {v:6.2f}  (spending from {X[col].min():.1f} to {X[col].max():.1f})")
```

Out [19]:

```
Average budget in the data:
TV           147.04
Radio         23.26
Newspaper     30.55
```

Scenario	TV	Radio	Newspaper	Predicted Sales	Change vs baseline
Average budget (baseline)	147.04	23.26	30.55	15.56	0.00
A. Add 15 to TV	162.04	23.26	30.55	16.35	0.79
B. Add 15 to Radio	147.04	38.26	30.55	17.42	1.86
C. Add 15 to Newspaper	147.04	23.26	45.55	15.63	0.07
D. Move 15 from Newspaper to Radio	147.04	38.26	15.55	17.35	1.79

5 rows × 5 columns

```
Sales gain from adding 15 to Radio when TV = 50 : 1.10
Sales gain from adding 15 to Radio when TV = 250: 2.36

Change in predicted Sales across the full range of each channel:
  TV         :  15.89  (spending from 0.7 to 296.4)
  Radio      :   5.28  (spending from 0.0 to 49.6)
  Newspaper  :   0.90  (spending from 0.3 to 114.0)
```

Three recommendations for the company

1. **Keep TV as the core of the budget.** TV has the strongest link with Sales (r = 0.90), and across its full spending range it moves predicted Sales by about 15.9 units (Radio 5.3, Newspaper 0.9). Adding 15 to TV at the average budget raises predicted Sales by about 0.79 units. Because the TV^2 term is negative, the extra return shrinks as TV spending gets very large, so increase TV spending gradually and check the result rather than expecting it to rise forever.
2. **Pair Radio with TV instead of treating them as separate choices.** Per unit of spending Radio adds more than TV (+1.86 against +0.79 for an extra 15), but its spending range in the data is small, so it supports TV rather than replaces it. The positive TV×Radio term shows that the two work better together: adding 15 to Radio brings about 1.10 extra Sales units when TV spending is low (50) but about 2.36 when TV spending is high (250).
3. **Stop adding to Newspaper and move that money to Radio.** Newspaper has the weakest link with Sales (r = 0.16) and its coefficient in the linear model is only 0.0043. Adding 15 to Newspaper changes predicted Sales by just +0.07 units, while moving 15 from Newspaper to Radio raises it by about 1.79 units at the same total spend.

Limitations: these figures come from a model fitted on only 200 records and show association, not proof of cause. They should be confirmed with a small test (for example, shifting part of the Newspaper budget to Radio in a few regions) before the whole budget is changed.

Result

Thus Linear Regression, Polynomial Regression (degree 2) and Support Vector Regression (RBF kernel) models were built on 80% of the Advertising dataset (200 records) and tested on the remaining 20%. TV was the feature most strongly correlated with Sales (r = 0.901). The Polynomial Regression (deg 2) gave the lowest prediction error (RMSE 1.2011, R^2 0.9533) and was selected as the best model. Based on it, three recommendations were made: keep TV as the core channel, pair Radio with TV, and move spending from Newspaper to Radio.